PHIẾU THỰC HÀNH — 09/10/2026

Họ tên: TRẦN HỮU DUY **MSSV:** N24DCDT022 **Lớp:** D24CQDT01-N

GitHub: `https://github.com/TranHuuDuy-PTIT/Thuc_hanh_Julia_2026_10_09` | **Commit SHA:** `9a91d0d`

---

1. Mô tả một luồng ứng dụng

Mục tiêu, đầu vào, đầu ra và sơ đồ truyền dữ liệu giữa các chức năng:

- **Mục tiêu:** Điều phối giao từng đơn hàng độc lập từ kho (đỉnh 1) tới điểm giao rồi quay về kho trong khuôn viên trường bằng ngôn ngữ Julia. Lựa chọn tập đơn hàng tối ưu trong giới hạn ngân sách thời gian *B* sao cho tổng lợi ích đạt được lớn nhất (mô hình Bài toán Ba lô 0/1).
- **Đầu vào:** Hai tệp CSV chuẩn hóa: `don_hang.csv` (id, diem_giao, thoi_gian_phuc_vu, loi_ich) và `ban_do.csv` (u, v — 7 cạnh nối vô hướng giữa 6 đỉnh khuôn viên).
- **Đầu ra:** Tập các đơn hàng tối ưu, tuyến đường di chuyển ngắn nhất cho mỗi đơn, chuỗi trạng thái điều phối giao và hoàn tác trên Hàng đợi vòng (`CircularQueue`) và Ngăn xếp (`Stack`), cùng tệp kết quả `ket_qua.csv` (9 cột chuẩn).
- **Sơ đồ truyền dữ liệu:**
  `[don_hang.csv, ban_do.csv]` → `read_data` (kiểm tra định dạng) → `merge_sort` (chuẩn hóa danh mục) → `bfs_routes` (đường ngắn nhất & chi phí $\tau = 2h + s$) → `knapsack` (quy hoạch động chọn đơn tối ưu $\leq B$) → `CircularQueue & Stack` (điều phối giao deliver! & hoàn tác undo!) → `export_result` (xuất ket_qua.csv).

2. Bảng truy vết đường và thời gian (MINI001)

ID	Đường lượt đi	Số cạnh h	2h + phục vụ
1	1 → 2	1	2 × 1 + 1 = **3**
2	1 → 2 → 4	2	2 × 2 + 1 = **5**
3	1 → 2 → 4 → 6	3	2 × 3 + 1 = **7**

3. Minh chứng bốn hàm

NV1 · Hoàn thiện merge_sort(a, before)

- **Ý tưởng/điều kiện áp dụng:** Áp dụng chiến lược Chia để trị (Divide and Conquer). Đệ quy chia đôi mảng đến khi kích thước ≤ 1, sau đó trộn (merge) hai nửa đã sắp xếp bằng hàm so sánh `before`. Đảm bảo tính ổn định (stable): khi hai phần tử bằng nhau (hòa) thì phần tử bên trái luôn được giữ trước. Áp dụng cho mọi mảng đối tượng có quan hệ thứ tự.
- **Lời gọi và kết quả kiểm thử:** `julia TH_2026_10_09_Kiem_thu.jl --phase=sort` → **ĐẠT 8/8 phép kiểm tra** (xử lý mảng rỗng, 1 phần tử, tăng/giảm dần, tính ổn định và comparator hòa).

- **Chi phí thời gian/bộ nhớ:** Thời gian *O(n log n)* trong mọi trường hợp; Bộ nhớ phụ *O(n)* do cấp phát mảng tạm khi trộn.

NV2 · Hoàn thiện binary_search(a::Vector{Order}, id::Int)

- **Ý tưởng/điều kiện áp dụng:** Tìm kiếm theo nguyên lý chia đôi khoảng tìm kiếm [lo, hi]. Tại mỗi bước tính `mid = lo + div(hi-lo, 2)` và so sánh `id` với `a[mid].id` để thu hẹp một nửa không gian. **Tiền điều kiện:** Mảng *a* phải được sắp xếp tăng dần theo trường `id`. Trả về đối tượng `Order` nếu thấy, hoặc `nothing` nếu không có.
- **Lời gọi và kết quả kiểm thử:** `julia TH_2026_10_09_Kiem_thu.jl --phase=search` → **ĐẠT 5/5 phép kiểm tra** (tìm thấy ở đầu, giữa, cuối; trả về nothing khi không tồn tại).
- **Chi phí thời gian/bộ nhớ:** Thời gian *O(log n)*; Bộ nhớ phụ *O(1)* (cài đặt lặp tối ưu).

NV3 · Hoàn thiện bfs_routes(graph, source=1)

- **Ý tưởng/điều kiện áp dụng:** Sử dụng hàng đợi vòng `CircularQueue` để duyệt theo chiều rộng (BFS) từ kho 1 trên đồ thị vô hướng không trọng số. Khởi tạo distance = -1, đỉnh nguồn distance[source] = 0. Khi duyệt u, xét đỉnh kề v chưa thăm, gán distance[v] = distance[u] + 1, parent[v] = u và đưa vào hàng đợi. Sau đó truy ngược parent từ đích về kho để dựng tuyến đường ít cạnh nhất.
- **Lời gọi và kết quả kiểm thử:** `julia TH_2026_10_09_Kiem_thu.jl --phase=bfs` → **ĐẠT 7/7 phép kiểm tra** (khoảng cách 6 đỉnh, parent, tính hợp lệ của tuyến đường).
- **Chi phí thời gian/bộ nhớ:** Thời gian *O(V + E)* (với V = 6, E = 7); Bộ nhớ phụ *O(V)*.

NV4 · Hoàn thiện knapsack(orders, costs, budget)

- **Ý tưởng/điều kiện áp dụng:** Quy hoạch động giải bài toán Ba lô 0/1. Bảng `table[i+1, b+1]` lưu lợi ích tối ưu khi xét *i* đơn đầu với ngân sách *b*. Hệ thức: `table[i+1, b+1] = table[i, b+1]` (không lấy); nếu `b ≥ c_i` và `table[i, b-c_i+1] + v_i > table[i+1, b+1]` thì chọn đơn *i*. Khi hòa lợi ích giữ phương án không chọn. Truy vết ngược từ `table[n+1, budget+1]`.
- **Lời gọi và kết quả kiểm thử:** `julia TH_2026_10_09_Kiem_thu.jl --phase=dp` → **ĐẠT 12/12 phép kiểm tra** (khớp tuyệt đối với thuật toán vét cạn `brute_force`).
- **Chi phí thời gian/bộ nhớ:** Thời gian *O(n × B)*; Bộ nhớ phụ *O(n × B)* cho bảng quy hoạch động.

4. Quyết định chọn đơn

- **Trên bộ dữ liệu mẫu MINI001:** Ngân sách *B = 8*. Tập đơn chọn: **[1, 2]**. Tổng thời gian: 3 + 5 = 8 (≤ 8). Tổng lợi ích: 6 + 8 = **14**.
  - **Giải thích ô DP F(2, 8)** (`table[3, 9]` trong Julia): Đơn 2 có c2 = 5, v2 = 8. Phương án 1 (không lấy đơn 2) nhận F(1, 8) = 6. Phương án 2 (lấy đơn 2) cho F(1, 8 - 5) + 8 = F(1, 3) + 8 = 6 + 8 = 14. Do 14 > 6 nên ô nhận giá trị **14**.
  - **Bước truy vết:** Tại i = 2, b = 8: table[3, 9] = 14 ≠ table[2, 9] = 6 → Chọn đơn 2, ngân sách còn lại 8 - 5 = 3. Tại i = 1, b = 3: table[2, 4] = 6 ≠ table[1, 4] = 0 → Chọn đơn 1. Tập chọn:

[1, 2].

- **Đối chiếu vét cạn:** Xét toàn bộ $2^3 = 8$ tập con; phương án {1, 2} có tổng chi phí 8, lợi ích 14 là tối ưu nhất. DP và vét cạn trùng khớp 100%.

- **Mở rộng trên bộ dữ liệu cá nhân N24DCDT022:** Ngân sách $B = 25$. Tập chọn gồm 5 đơn: **[4, 5, 6, 7, 2]**. Tổng thời gian: 3 + 5 + 7 + 5 + 5 = 25. Tổng lợi ích tối ưu: 12 + 13 + 13 + 9 + 6 = **53** (trùng khớp tuyệt đối giữa DP, vét cạn và tham lam).

5. Giao và hoàn tác

Trạng thái	Queue (Chờ)	Stack (Đã giao)	Đã dùng	Đã nhận
Sau lập kế hoạch	[1, 2]	[]	0	0
Sau giao một đơn	[2]	[1]	3	6
Sau hoàn tác	[1, 2]	[]	0	0
Sau giao hết	[]	[1, 2]	8	14

6. Đầu ra và nhận xét

- **Đường dẫn CSV kết quả:** `data/MINI001/ket_qua.csv` và `data/N24DCDT022/ket_qua.csv`
- **Link notebook:** `TH_2026_10_09_Pluto.jl`
- **Lỗi đã phát hiện và cách sửa:**
  1. *Lệch chỉ số mảng 1-based trong Julia:* Bảng DP kích thước (n+1) × (budget+1), trạng thái (i, b) phải truy cập ở `table[i+1, b+1]`.
  2. *Tra cứu chi phí:* Tham số costs là Dict ánh xạ theo ID, phải tra bằng `costs[orders[i].id]` thay vì `costs[i]`.
  3. *Thứ tự duyệt kề:* Sử dụng `merge_sort` để sắp xếp danh sách kề theo `tie` cá nhân hóa giúp kết quả BFS ổn định và tái lập được.
- **Một chức năng thay đổi ảnh hưởng đến chức năng sau:** Các chức năng tạo thành đường ống dữ liệu liên hoàn: Nếu `bfs_routes` tính sai đường dẫn đến chi phí τ sai → bảng quy hoạch động `knapsack` nhận chi phí sai và chọn ra tập đơn vượt ngân sách hoặc sai lợi ích → hàng đợi và ngăn xếp mô phỏng giao nhận sai → tệp kết quả `ket_qua.csv` sai toàn bộ.

Câu hỏi cuối buổi (Q1 – Q5)

Q1. Vì sao không được dùng dãy đã sắp giảm theo lợi ích/τ để tìm kiếm nhị phân theo ID? Nêu tiền điều kiện của hàm tìm kiếm.

Trả lời: Dãy sắp giảm theo lợi ích/τ được sắp xếp theo tỷ số lợi ích/chi phí nhằm phục vụ giải thuật tham lam, không bảo toàn tính đơn điệu theo trường ID. Tiền điều kiện bắt buộc của tìm kiếm nhị phân theo ID là mảng đầu vào phải được sắp xếp tăng dần (hoặc đơn điệu) theo trường ID. Nếu mảng chưa sắp theo ID, điều kiện so sánh giá trị giữa `mid` với ID cần tìm sẽ loại bỏ sai nửa không gian chứa phần tử, khiến hàm trả về `nothing` dù ID thực sự có trong danh mục.

Q2. Với đơn ID 2 tới điểm 4, phục vụ 1 và bản đồ mẫu, giải thích từng thành phần của thời gian thực hiện τ. BFS có tiếp tục cho thời gian ngắn nhất nếu các cạnh mang thời gian khác nhau không?

Trả lời: Với đơn ID 2 giao tới điểm 4, phục vụ 1: Đường đi ngắn nhất từ kho 1 tới đỉnh 4 trên bản đồ là 1 → 2 → 4 (số cạnh h = 2). Thời gian τ gồm: lượt đi từ kho tới điểm giao (h = 2), lượt quay về kho (h = 2), và thời gian phục vụ tại chỗ s = 1. Tổng thời gian: τ = 2h + s = 2×2 + 1 = **5**. Nếu các cạnh mang thời gian khác nhau (đồ thị có trọng số), BFS **KHÔNG CÒN** đảm bảo tìm được đường đi ngắn nhất nữa, vì BFS mở rộng theo số lượng cạnh ít nhất chứ không xét tổng trọng số thời gian. Khi đó bắt buộc phải chuyển sang thuật toán Dijkstra.

Q3. Giải thích hai phương án tạo ô F(2,8) và vị trí của ô đó trong table Julia. Vì sao nhánh lấy đơn phải dùng hàng trước?

Trả lời: Trong Julia, do chỉ số bắt đầu từ 1 nên ô F(2, 8) nằm ở tọa độ `table[2+1, 8+1] = table[3, 9]`. Hai phương án tạo ô: (1) Không chọn đơn 2: Kế thừa giá trị tối ưu của 1 đơn đầu với ngân sách 8: F(1, 8) = 6; (2) Chọn đơn 2: Nhận lợi ích v2 = 8 cộng với lợi ích tối ưu của 1 đơn đầu với ngân sách còn lại (8 - c2 = 8 - 5 = 3): F(1, 3) + 8 = 6 + 8 = 14. Chọn max(6, 14) = **14**. Nhánh lấy đơn bắt buộc phải dùng hàng trước (i - 1) vì đây là bài toán ba lô 0/1 (mỗi đơn chỉ được chọn tối đa một lần); nếu dùng cùng hàng i thì một đơn hàng có thể bị cộng dồn nhiều lần (trở thành bài toán ba lô không giới hạn).

Q4. Sau khi giao ID 1 rồi hoàn tác ngay, FIFO, ngăn xếp, tổng thời gian và tổng lợi ích phải thay đổi thế nào? Nêu một bất biến kiểm tra việc không mất hoặc lặp đơn.

Trả lời: Khi giao ID 1: Hàng đợi FIFO lấy đơn 1 ra khỏi đầu (`dequeue!`), ngăn xếp Stack đẩy đơn 1 vào đỉnh (`push!`), tổng thời gian tăng 3 (lên 3), tổng lợi ích tăng 6 (lên 6). Khi hoàn tác ngay: Ngăn xếp lấy đơn 1 ra khỏi đỉnh (`pop!`), chèn ngược lại vào đầu hàng đợi FIFO (`enqueue_front!`), tổng thời gian giảm 3 (về 0), tổng lợi ích giảm 6 (về 0). Hệ thống quay về nguyên vẹn trạng thái ban đầu. Bất biến kiểm tra: (1) Bảo toàn tập đơn: `set(Queue) ∪ set(Stack) = set(selected_ids)` ; (2) Không lặp đơn: `set(Queue) ∩ set(Stack) = ∅` ; (3) Chi phí và lợi ích đã dùng/đã nhận khớp chính xác với tổng của các đơn nằm trong Stack.

Q5. Cho biết độ phức tạp theo n,V,E,B của sắp xếp trộn, tìm kiếm nhị phân, BFS cốt lõi và DP bảng đầy đủ. Vì sao không thể khẳng định mọi BST đều tra cứu trong O(log n)?

Trả lời: Độ phức tạp: Sắp xếp trộn: thời gian *O(n log n)*, bộ nhớ *O(n)*; Tìm kiếm nhị phân: thời gian *O(log n)*, bộ nhớ *O(1)*; BFS cốt lõi: thời gian *O(V + E)*, bộ nhớ *O(V)*; DP bảng đầy đủ: thời gian *O(n × B)*, bộ nhớ *O(n × B)*. Không thể khẳng định mọi BST đều tra cứu trong *O(log n)* vì thời gian tìm kiếm trên BST phụ thuộc vào chiều cao *h* của cây (*O(h)*). Trong trường hợp xấu nhất khi các khóa chèn theo thứ tự tăng/giảm dần, cây bị suy biến thành danh sách liên kết lệch một phía với chiều cao *h = O(n)*, chi phí tìm kiếm suy biến thành *O(n)*. Cây chỉ đạt *O(log n)* khi được giữ cân bằng (như AVL Tree hoặc Red-Black Tree).